

WEEK 9 PRACTICAL

QUESTION – 1: Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

AIM

To implement a file copy utility using low-level Linux file I/O system calls and compare it with standard C library file I/O functions.

PROGRAM

Part A — Low-Level File I/O

Step 1: Create the Week 9 folder

```
ashborn@DESKTOP-I4G5Q4N:~$ mkdir week9_p
ashborn@DESKTOP-I4G5Q4N:~$ cd ~/week9_p
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ pwd
/home/ashborn/week9_p
ashborn@DESKTOP-I4G5Q4N:~/week9_p$
```

Step 2: Create the source file

```
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ echo "This is Week 9 OSSP file copy practical." > source.txt
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ cat source.txt
This is Week 9 OSSP file copy practical.
```

Step 3: Create the C program

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int src, dest;
    char buffer[1024];
    ssize_t bytes;
    off_t size;

    src = open("source.txt", O_RDONLY);

    if (src < 0) {
        perror("Error opening source file");
        return 1;
    }

    dest = open("copy.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest < 0) {
        perror("Error creating destination file");
        close(src);
        return 1;
    }

    size = lseek(src, 0, SEEK_END);
    printf("Source file size: %ld bytes\n", size);

    lseek(src, 0, SEEK_SET);

    while ((bytes = read(src, buffer, sizeof(buffer))) > 0) {
        write(dest, buffer, bytes);
    }

    close(src);
    close(dest);

    printf("File copied successfully using low-level I/O.\n");

    return 0;
}
```

```
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ nano filecopy.c
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ gcc filecopy.c -o filecopy
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ ./filecopy
Source file size: 41 bytes
File copied successfully using low-level I/O.
```

Part B — Standard Library Functions

Step 5: Create the standard I/O program

```
#include <stdio.h>

int main() {
    FILE *src, *dest;
    char buffer[1024];
    size_t bytes;

    src = fopen("source.txt", "r");
    dest = fopen("copy_stdio.txt", "w");

    if (src == NULL || dest == NULL) {
        perror("Error opening files");
        return 1;
    }

    while ((bytes = fread(buffer, 1, sizeof(buffer), src)) > 0) {
        fwrite(buffer, 1, bytes, dest);
    }

    fclose(src);
    fclose(dest);

    printf("File copied successfully using standard I/O.\n");

    return 0;
}
```

```
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ nano filecopy_stdio.c
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ gcc filecopy_stdio.c -o filecopy_stdio
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ ./filecopy_stdio
File copied successfully using standard I/O.
```

Step 6: Compare both files

```
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ cmp copy.txt copy_stdio.txt
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ ls -l source.txt copy.txt copy_stdio.txt
-rw-r--r-- 1 ashborn ashborn 41 Sep 28 05:16 copy.txt
-rw-r--r-- 1 ashborn ashborn 41 Sep 28 05:19 copy_stdio.txt
-rw-r--r-- 1 ashborn ashborn 41 Sep 28 05:14 source.txt
```

Step 7: Optional performance comparison

```
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ time ./filecopy
Source file size: 41 bytes
File copied successfully using low-level I/O.

real    0m0.014s
user    0m0.000s
sys     0m0.000s
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ time ./filecopy_stdio
File copied successfully using standard I/O.

real    0m0.012s
user    0m0.000s
sys     0m0.016s
```

OBSERVATION

The file was successfully copied using both low-level system calls and standard C library functions. The low-level implementation directly used file descriptors with `open()`, `read()`, `write()`, `lseek()` and `close()`, while the standard I/O implementation used `fopen()`, `fread()`, `fwrite()` and `fclose()`. Both methods produced identical copies.

RESULT

The file copy utility was successfully implemented using low-level Linux file I/O system calls and compared with standard library file I/O functions.

QUESTION – 1: Develop a program that redirects standard input and output using file descriptors and `dup2()`. Demonstrate how shells implement I/O redirection.

AIM

To implement standard input/output redirection using file descriptors and the dup2() system call and demonstrate shell I/O redirection.

PROGRAM

Step 1: Create the program, Compile and Run

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;

    fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0) {
        perror("Error opening file");
        return 1;
    }

    dup2(fd, STDOUT_FILENO);

    close(fd);

    printf("This output is redirected to output.txt\n");

    return 0;
}
```

Step 2: Check the output

```
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ nano redirect.c
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ gcc redirect.c -o redirect
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ ./redirect
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ cat output.txt
This output is redirected to output.txt
```

Demonstrate Shell I/O Redirection

Step 3: Output redirection using >

```
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ echo "Hello from shell redirection" > shell_output.txt
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ cat shell_output.txt
Hello from shell redirection
```

Step 4: Input redirection using <

```
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ echo "Input redirection demonstration" > input.txt
ashborn@DESKTOP-I4G5Q4N:~/week9_p$ cat < input.txt
Input redirection demonstration
```

OBSERVATION

The program successfully redirected standard output from the terminal to a file using `dup2()`. Shell redirection was also demonstrated using `>` for output and `<` for input.

RESULT

Standard input and output redirection was successfully implemented using file descriptors and `dup2()`, and the working of shell I/O redirection was demonstrated.